BroStatus v1.0.0 by DiamondSash

Die Statusanzeige für Claude Code: Ein Diamant zeigt dir mit einer Farbe, ob Claude arbeitet, auf dich wartet oder fertig ist. Daneben stehen Kontext, Prompt-Cache, genutzte Werkzeuge und geänderte Dateien, dazu Antwort-Knöpfe über dem Eingabefeld.

Stand 04.11.2026 · Version 1.0.0 · getestet mit Claude Code 2.1.310 · Windows | Mac | Linux · ohne eigene Server

Wo es läuft: nur in Claude Code – im Terminal (`claude`) unter Windows, Mac und Linux, in der Claude-Desktop-App im Bereich „Code“ und in Editoren wie VS Code mit der Claude-Code-Erweiterung. **Getestet** (eigene Tests): Windows 10 mit der Desktop-App im echten Einsatz, VS Code unter Windows (29.10.2026: Panel, Status, Währung), Installer auf einem zweiten Windows-PC. Die Fassung ohne die automatischen Funktionen habe ich in der echten Oberfläche noch nicht angesehen; sie teilt den Code mit der vollen Fassung, die ich am 30.10.2026 im Terminal (PowerShell) und in der Desktop-App getestet habe (Diamant, Panel). Mac: von einer Person im Terminal ausprobiert, Desktop-App ungetestet. Linux: Installer liegt bei, in der Praxis noch ungetestet, Rückmeldungen willkommen. Im normalen Claude-Chat und in anderen KI-Werkzeugen (zum Beispiel Cursor, Codex, Gemini) läuft es nicht.

5 Farben
zeigt der Diamant: kritisch, du bist dran, bald handeln, arbeitet, fertig – die wichtigste gewinnt

3 Wege
dich zu holen: Fenstertitel, Meldung im Terminal, Telegram aufs Handy

0 Tokens
kostet die Anzeige selbst (ein Token ist die Einheit, in der Claude Text zählt und abrechnet)

Was du davon hast

Was BroStatus für dich übernimmt.

Wann du dran bist

Der Diamant wird grau, sobald Claude auf dich wartet. Bei längeren Läufen meldet sich der Rechner oder das Handy.

Der Diamant · Signale

Cache und Kosten

Du siehst den Cache-Countdown. Ist der Cache kalt, siehst du es und weißt, warum die nächste Nachricht mehr kostet. Willst du den Cache behalten, schick rechtzeitig selbst eine Nachricht.

Cache-Zeile

Kontext sehen

Die Ampel warnt rechtzeitig, bevor der Kontext voll ist. Dann fasst du selbst mit `/compact` zusammen oder fängst einen neuen Chat an.

Kontext-Ampel

Fragen beantworten

Pro Punkt ein Knopf, Antwort tippen, alles gesammelt senden. Claude bekommt jede Antwort mit der passenden Frage.

Antwort-Knöpfe

Was Claude tut

Live zeigt BroStatus, welche Werkzeuge laufen und welche Dateien geändert, neu oder nur gelesen sind.

Aktiv und Dateien

Beispiel: Cache

Du arbeitest an etwas, wirst abgelenkt und vergisst den Chat. Der Cache, der Zwischenspeicher für den schon gelesenen Teil des Gesprächs, läuft nach einer Pause ab. BroStatus zeigt dir den Countdown. Ist der Cache kalt, siehst du es und weißt, warum die nächste Nachricht mehr kostet. Dein Gespräch geht dabei nicht verloren. Willst du den Cache behalten, schick rechtzeitig selbst eine Nachricht.

Beispiel: Compact

Du programmierst in einem Fluss und siehst den Kontext nicht. Je voller er wird, desto mehr lassen Genauigkeit und Erinnerung nach (oft „Context Rot“ genannt). Der Kontext-Balken zeigt dir den Füllstand, die Ampel warnt rechtzeitig. Dann kannst du selbst mit `/compact` zusammenfassen oder einen neuen Chat anfangen. Was in deinen Dateien steht, bleibt erhalten, was nur im Chat stand, steckt danach nur noch in der kürzeren Zusammenfassung.

Beispiel: Töne

Du machst etwas anderes, während Claude arbeitet. Fragt Claude nach einer Freigabe oder zeigt eine Auswahl, spielt BroStatus in der Desktop-App einen Warnton, und du kannst hingehen und reagieren. Ist Claude nach längerer Arbeit fertig, kommt ein leiser Ton. Beides stellst du unter „Mehr“ ein oder schaltest es aus. Im Terminal piept stattdessen die Terminal-Glocke.

In dieser Fassung nicht enthalten: Cache warmhalten, Auto-Compact, Feierabend und Modellwechsel. Diese Funktionen senden von selbst Anfragen über dein Konto. Ob das zu den Bedingungen der Anthropic-Pläne passt, ist bei Anthropic angefragt und noch nicht beantwortet; sie kommen erst, wenn das geklärt ist.

Läuft lokal und kostenlos: Die Anzeige läuft ohne eigene Server, sendet nichts an den Autor und verbraucht keine Tokens. Was Claude Code ohnehin an Anthropic überträgt, bleibt, wie es ist. Darüber hinaus verlässt nur Folgendes deinen Rechner: Telegram (nur wenn du es selbst einrichtest) und unter Windows der Python-Download über winget (der Installer stimmt dabei den Bedingungen von winget und Python für dich zu). Im Ordner `state/` liegen lokale Statusdateien mit Dateipfaden und Befehlsanfängen. BroStatus schickt von sich aus keine Anfragen über dein Konto.

Inhalt

Alle Bilder der Statusline kommen direkt aus BroStatus selbst.

01 Auf einen Blick
02 Der Diamant
03 Statuszeile
04 Kontext-Ampel
05 Cache-Zeile
06 Aktiv und Dateien
07 Antwort-Knöpfe
08 Das /bro-Panel
09 Fertig- und Warte-Signale
10 Installation
11 Prüfen, ob alles läuft
12 Einstellungen
13 Updates und Probleme lösen

Auf einen Blick

So sieht BroStatus unten in Claude Code aus, während Claude arbeitet.



Echte Ausgabe der Statusline mit Beispieldaten: Claude arbeitet seit 1 Minute 12 Sekunden und führt gerade Tests aus.

DIAMANT	Links steht das Signet. Seine Farbe zeigt in einer Sekunde, ob du gebraucht wirst. Mehr dazu
STATUS	Was Claude gerade tut, wie lange schon, welches Werkzeug läuft, dazu Modell und Denkstufe. Mehr dazu
KONTEXT	Wie voll das Gedächtnis der Unterhaltung ist, mit Ampel und Tipp, wann du aufräumen solltest. Mehr dazu
CACHE	Wie lange der Prompt-Cache noch warm ist, wie gut er trifft und wie viel von deinen Limits verbraucht ist. Mehr dazu
AKTIV	Welche Werkzeuge, Skills, MCP-Server, Webadressen und Hilfs-Agenten in dieser Sitzung benutzt wurden. Mehr dazu
DATEIEN	Welche Dateien Claude geändert, neu angelegt oder nur gelesen hat. Mehr dazu

Der Diamant

Immer genau eine Farbe. Trifft mehreres zu, gewinnt die wichtigste.



Rot · kritisch

API-Fehler, Kontext ab 85 % oder der Cache läuft in den nächsten 2 Minuten (1-h-Cache) bzw. 30 Sekunden (5-min-Cache) ab.



Grau · du bist dran

Claude wartet auf deine Freigabe oder Antwort. Hell-silbern, damit es auffällt.



Orange · bald handeln

Kontext ab 50 % oder der Cache läuft bald ab (1-h-Cache: unter 10 Minuten, 5-min-Cache: unter 1 Minute).



Blau · arbeitet

Claude ist mitten in einer Antwort. Du musst nichts tun.



Grün · fertig

Antwort ist da, alles in Ordnung. Auch im Ruhezustand „bereit".

Rangfolge: rot › grau › orange › blau › grün

Beispiel: Claude arbeitet, aber der Kontext steht bei 58 %. Dann ist der Diamant orange statt blau, weil „bald handeln" wichtiger ist als „arbeitet". Die Kontext-Grenzen änderst du über `ctx_stages` in den Einstellungen.

Statuszeile

Die erste Zeile sagt, was Claude gerade tut.

BEREIT

Noch nichts passiert in dieser Sitzung. Rechts stehen Modell und Denkstufe, hier Opus mit „high".

```
○ bereit · opus/high
```

ARBEITET

Claude arbeitet seit 1 Minute 12 Sekunden. Hinter dem Pfeil steht das laufende Werkzeug mit seiner Laufzeit. Laufen mehrere gleichzeitig, steht „+2 parallel" dahinter.

```
⚙ arbeitet 1m 12s → Bash: npm test 8s · opus/high
```

DENKT NACH

Claude arbeitet, benutzt aber gerade kein Werkzeug.

```
⚙ arbeitet 5s → denkt nach… · opus/high
```

WARTET

Claude braucht dich, meistens für eine Freigabe. Dahinter steht, worauf Claude wartet. Der Diamant wird grau.

```
‖ wartet auf dich – Bash: git push · opus/high
```

FERTIG

Die Antwort ist da: vor 12 Sekunden fertig geworden, nach 3 Minuten 20 Sekunden Arbeit.

```
✓ fertig vor 12s · lief 3m 20s · opus/high
```

IM HINTERGRUND

Claude hat geantwortet, aber ein Befehl oder Hilfs-Agent läuft noch, hier zwei. Statt „fertig" steht deshalb „arbeitet im Hintergrund" mit der ersten Aufgabe, der Diamant bleibt blau. Im Panel steht darunter je Aufgabe eine Zeile. Erst wenn alles durch ist, kommt „fertig" mit Ton.

```
◔ arbeitet im Hintergrund · 2 → npm run build +1 · vor 40s · opus/high
```

ABGEBROCHEN

Die Antwort wurde durch einen Fehler der Schnittstelle abgebrochen. Meist hilft es, die Nachricht noch einmal zu schicken.

```
× abgebrochen (API-Fehler) vor 3s · opus/high
```

Kontext-Ampel

Wie voll das Gedächtnis der Unterhaltung ist, mit Tipp, was jetzt zu tun ist.

UNTER 50 %

Cyan: alles entspannt. Hinter der Prozentzahl stehen die genutzten und die möglichen Tokens.

```
kontext ▮▮▮▮▮▯▯▯▯▯▯▯▯▯ 30% 60k/200k
```

AB 50 %

Gelb: **Aufgabe abschließen**. Mach die laufende Aufgabe fertig, bevor du etwas Neues anfängst.

```
kontext ▮▮▮▮▮▮▮▮▮▯▯▯▯▯▯ 58% 116k/200k ▸ Aufgabe abschließen
```

AB 70 %

Orange: **/compact oder neuer Chat**. Jetzt ist ein guter Moment, aufzuräumen.

```
kontext ▮▮▮▮▮▮▮▮▮▮▮▮▯▯▯▯ 75% 150k/200k ▸ /compact oder neuer Chat
```

AB 85 %

Rot: **gleich zusammenfassen!** Claude Code kompaktiert bald von selbst.

```
kontext ▮▮▮▮▮▮▮▮▮▮▮▮▮▮▯▯ 88% 176k/200k ▸ gleich Auto-Compact!
```

GUTER MOMENT

Ist der Cache schon kalt und Claude gerade nicht beschäftigt, kostet ein `/compact` kaum mehr als der ohnehin fällige Neuaufbau. Dann erscheint der grüne Hinweis.

```
kontext ▮▮▮▮▮▮▮▮▮▮▮▮▯▯▯▯ 75% 150k/200k ▸ /compact oder neuer Chat (Cache kalt → guter Moment)
```

Cache-Zeile

Der Prompt-Cache spart Geld und Kontingent, solange er warm ist.

Dein Vorteil: Du weißt, wie lange du Pause machen kannst, ohne dass die nächste Nachricht teuer wird.

Der bisherige Verlauf einer Unterhaltung wird bei Anthropic für eine Weile zwischengespeichert. Solange dieser Cache warm ist, wird der Verlauf bei deiner nächsten Nachricht nur gelesen, und das kostet etwa ein Zehntel (je nach Modell weniger). Ist er abgelaufen, muss alles neu geschrieben werden.

WARM

Der Countdown zeigt, wie lange der Cache noch hält, daneben die Dauer (`1h` oder `5m`).

```
cache  ◔ 54:58 1h  treffer 99% letzte · 90% sitzung  limit · S | 5h / 23%  · W | 7d / 71%  #tools_chan…
```

LÄUFT BALD AB

Gelb, sobald beim 1-h-Cache weniger als 10 Minuten übrig sind. Beim 5-min-Cache gibt es hier kein Gelb, nur im Panel.

```
cache   ◷ 8:05 1h · läuft bald ab   treffer 99% letzte · 90% sitzung   limit · S | 5h / 23%  · W | 7d / 71…
```

GLEICH WEG

Rot in der Cache-Zeile: in den letzten 2 Minuten, beim 5-min-Cache in der letzten Minute (Diamant und Panel werden beim 5-min-Cache erst in den letzten 30 Sekunden rot). Wenn du weitermachen willst, schick jetzt deine Nachricht.

```
cache   ◷ 1:34 1h · läuft bald ab   treffer 99% letzte · 90% sitzung   limit · S | 5h / 23%  · W | 7d / 71…
```

KALT

Abgelaufen. „neu ~92k" heißt: Deine nächste Nachricht schreibt etwa 92.000 Tokens neu in den Cache.

```
cache   ◷ kalt 1h · neu ~92k   treffer 99% letzte · 90% sitzung   limit · S | 5h / 23%  · W | 7d / 71%   #…
```

5-MIN-CACHE

Je nach Konto und Einstellung hält der Cache nur 5 Minuten (oft bei API-Schlüssel oder Zusatz-Guthaben).

```
cache   ◷ 3:31 5m   treffer 99% letzte · 90% sitzung   limit · S | 5h / 23%  · W | 7d / 71%   #tools_chang…
```

Treffer

- **letzte:** Anteil der letzten Anfrage, der aus dem Cache kam
- **sitzung:** dasselbe für die ganze Sitzung
- Hoch ist gut. 99 % heißt: fast alles wurde billig gelesen

Limit

- Verbrauch deines Abos: **5-Stunden-Fenster / Woche**
- Weiß bis 64 %, gelb ab 65 %, rot ab 85 %

#Gründe

- Warum der Cache zuletzt neu geschrieben werden musste
- `#tools_changed×2` : Die Werkzeugliste hat sich zweimal geändert

Aktiv und Dateien

Was in dieser Sitzung benutzt und angefasst wurde.

```
aktiv   read×6  edit×3  bash×2  grep×2  ◇ pdf  ⌁ github×2  ↗ api.github.com  ⬡ Explore
```

Zeichen in der Aktiv-Zeile

- **ohne Zeichen:** Werkzeuge wie read, edit, bash, mit Anzahl (`read×6`)
- ◇ Skills, die Claude geladen hat
- ⌁ MCP-Server, also angebundene Dienste
- ↗ Webadressen und Websuche
- ⬡ Hilfs-Agenten, die mitgeholfen haben
- Pro Art stehen höchstens 5 Einträge, der Rest als `+3`

Zeichen bei den Dateien

- \+ geschrieben (neu oder überschrieben)
- ✎ geändert
- · nur gelesen
- Geänderte stehen vorne, jeweils die neuesten zuerst

```
dateien ✎ statusline.py  + hilfe.py  ✎ README.md  · anleitung.md  · config.json
```

Kompakte Ansicht (Standard): eine Zeile, was nicht passt, steht als `+2` am Ende.

```
├─ docs/
│  └─ · anleitung.md
├─ src/
│  ├─ neu/
│  │  └─ + hilfe.py
│  ├─ · config.json
│  └─ ✎ statusline.py
└─ ✎ README.md
```

Mit `"files": "tree"` erscheint darunter ein Dateibaum mit bis zu 8 Zeilen.

Antwort-Knöpfe

Mod „bro-status": offene Punkte beantworten, ohne alles abzutippen.

Endet Claudes Antwort mit einer nummerierten Liste oder Aufzählung, die etwas von dir wissen will, erscheint über dem Eingabefeld ein Band mit einem Knopf pro Punkt. Du beantwortest die Punkte einzeln, und „Send" schickt alles gesammelt als deine Nachricht.

```
◆ Antworten  1/2 beantwortet  [ Send (1) ]  [ Hide ]
[ Es fehlen noch 4 Bilder ]   ✓ sind schon im Ordner
[ Du musst das noch machen ]
Zu 2 ist erledigt                          Übernehmen
```

Nachgebaut nach dem Code der Mod. In Claude Code sieht das Band je nach Terminal leicht anders aus.

So geht's

1. Knopf des Punkts anklicken
2. Antwort ins Feld „Zu 1“ tippen, `Enter`. Es springt zum nächsten offenen Punkt
3. „Send“: alle Antworten gehen gesammelt raus
4. „Hide“ versteckt das Band bis zur nächsten Antwort

Was ankommt

Aus jeder Antwort wird eine Zeile mit Nummer und dem Punkt in Klammern, damit Claude genau weiß, worauf du dich beziehst:

```
Zu 1 (Es fehlen noch 4 Bilder): sind schon im Ordner
Zu 2 (Du musst das noch machen): ist erledigt
```

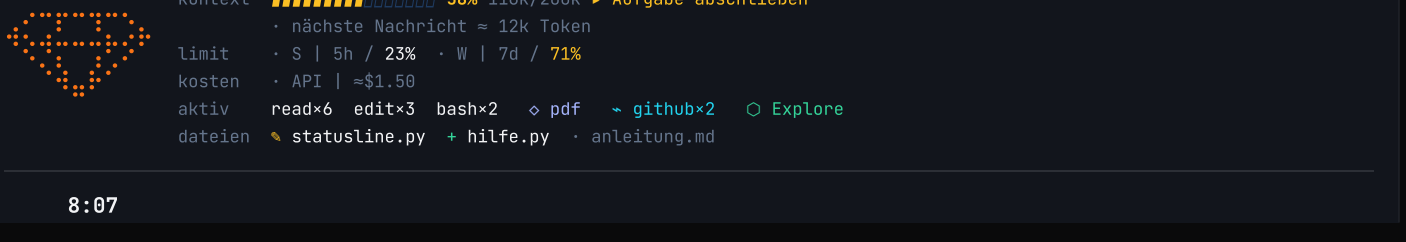
Gut zu wissen

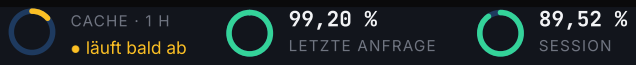
- Es zählt die letzte Liste mit mindestens 2 Punkten, höchstens 9 Knöpfe
- Nur bei Fragen: Ein „?“ steht in einem Punkt, in der Zeile direkt davor oder im Absatz direkt danach (bei Aufzählungen mit „-“ zählt nur der Absatz danach). Eine Überschrift direkt davor wie „Was ich von dir brauche:“ oder „Offene Fragen:“ zählt auch. Wörter wie „Gib mir Bescheid“ allein lösen kein Band aus, reine Anleitungen und Aufzählungen auch nicht
- Eine offene Frage ohne Auswahl („Wie soll ich das umsetzen?“, „Links oder rechts?“) bekommt kein Band, die beantwortest du direkt im Eingabefeld.
- Eine Ja/Nein-Frage am Ende (zum Beispiel „Passt der Plan so?“) bekommt zwei Knöpfe **Ja** und **Nein**. Ein Klick sendet sofort „Ja“ oder „Nein“, auch wenn eine Liste davor steht
- Nennt Claude Wege als eigene Zeilen („Weg 1 (empfohlen): …“, „Weg 2: …“) und fragt offen („Wie soll ich das umsetzen?“), gibt es je Weg einen Knopf **WEG 1**, **WEG 2**; die Empfehlung ist markiert, ein Klick sendet sofort „Weg 1“
- Unter einem Bericht ohne Frage stehen nach einer fertigen Aufgabe die Knöpfe **NÄCHSTE AUFGABE**, **SPEICHERN** und **WAS IST OFFEN?**, bei einem Zwischenstand nur **WAS IST OFFEN?**; kurze Antworten bekommen nichts. Erkannt wird über Wörter wie „geprüft“, „grün“ oder „folgt als Nächstes“. Gesendet werden „Zeig mir die startbaren Aufgaben.“, „Speichere jetzt: prüfen, committen, Pull Request.“ und „Was ist bei dieser Aufgabe noch offen?“. Gedacht sind sie für Projekte mit Aufgabenliste und Git
- Lange Punkte werden am Wortende gekürzt, die volle Frage steht im Antwortfeld
- Listen in Codeblöcken werden übersprungen
- Ohne Maus: `Strg+X`, dann `Tab` zu den Knöpfen und `Enter`. `Esc` geht zurück ins Eingabefeld
- Die Knöpfe haben bewusst keine Tastenkürzel: Beim Schreiben wird nie aus Versehen etwas ausgelöst
- Jede neue Nachricht leert das Band

Das /bro-Panel

Mit `/bro` öffnest du das Panel. In der Desktop-App öffnet es sich mit deiner ersten Nachricht von selbst.

Dein Vorteil: Auch in der Desktop-App, wo es keine Statusline gibt, siehst du alles auf einen Blick. Die drei Kreise zeigen die Cache-Restzeit, die letzte Anfrage und die Sitzung.





Töne
Warnton bei „wartet auf dich“, leiser Ton bei „fertig“

An

Mehr ▸

Nachgebaut nach dem Code der Mod, so wie die Desktop-App es zeigt. Oben steht der Status mit Diamant, darunter der Cache. Im Terminal stehen Restzeit und Trefferquoten als Text statt als Kreise. Die Zeile Töne zeigt links Name und Zustand, rechts den Schalter. Die Feineinstellungen liegen unter „Mehr“.

STATUS OBEN	Dieselben Angaben und Farben wie die Statusline: Diamant, was Claude tut, Kontext-Ampel, Limits und Kosten, genutzte Werkzeuge und Dateien. Gedacht für die Desktop-App, im Terminal ist es eine Zusammenfassung der Statusline.
WARTET AUF DICH	Bittet Claude Code dich um eine Freigabe oder zeigt dir Claude eine Auswahl zum Anklicken, steht sofort in roter Schrift „wartet auf dich“ mit dem, was freigegeben werden soll, und in der Desktop-App kommt der Warnton. Im Auto-Modus prüft zuerst der Auto-Prüfer: Solange er prüft, steht dort „Auto prüft: <Werkzeug>“ und es bleibt ruhig. „wartet auf dich“ kommt nur, wenn wirklich du gefragt wirst. Die Freigabe erkennt BroStatus über die Hooks aus `settings.json`; fehlen sie, steht „wartet auf dich“ erst, wenn das Werkzeug nach 30 Sekunden noch nicht gestartet ist. Die Desktop-App meldet die Freigabe nicht zurück: Die Anzeige bleibt, bis das Werkzeug fertig ist. Nach 30 Sekunden steht deshalb dahinter „Freigabe oder läuft seit …“ mit hochzählender Zeit, kurz vor und nach dem Ablauf des Caches zusätzlich eine Cache-Warnung.
CACHE GESCHÄTZT	Läuft die Statusline, kommt der Cache-Countdown genau aus ihren Daten. In der Desktop-App ohne Statusline schätzt BroStatus 1 Stunde ab der letzten Anfrage und schreibt „geschätzt“ dazu. Die Trefferquoten kommen dann aus Claudes Antworten.

KNOPF / FELD	WIRKUNG
◆▾ / ◆▸ (links vor den Kreisen)	Blendet Schalter und Mehr aus oder ein, damit das Panel weniger Platz braucht. Status und Kreise bleiben. Wird gemerkt.
Mehr ▸ / Mehr ▾	Klappt auf, in Abschnitten: die drei Kreise und die Limits, Töne, Sprache, Kosten und Tipp
Sprache (unter Mehr)	Deutsch, English, Español, Italiano oder Français. Gilt für Panel, Antwort-Band, Meldungen und die Statusline im Terminal; die Wahl bleibt gespeichert.
Währung (unter Mehr)	Dollar, Euro, Pfund oder Franken. Der API-Wert in der Zeile „kosten“ wird mit einem eingebauten Kurs umgerechnet (EZB-Referenzkurs, das Datum steht unter der Auswahl). Einen eigenen Kurs trägst du in `config.json` ein (`currency_rate` : wie viele Einheiten der Währung ein Dollar sind). Es bleibt eine Schätzung (≈), kein Rechnungsbetrag. Die Kurse stehen in `kurse.json` und werden mit neuen Versionen aktualisiert; BroStatus lädt sie nicht aus dem Netz.
Schließen	Noch einmal `/bro` oder `Esc`, in der Desktop-App das Kreuz am Panel. Alles Eingeschaltete läuft weiter
· S ⎪ 5h / … · W ⎪ 7d / … (Zeile „limit“)	S ist das Sitzungslimit (die laufenden 5 Stunden), W das Wochenlimit (7 Tage), jeweils wie viel davon verbraucht ist. Ab 65 % gelb, ab 85 % rot. In der Statusline gleich.
· API ⎪ $ (Zeile „kosten“)	API ist der geschätzte API-Wert der Sitzung (Claude Code rechnet ihn lokal aus den Token zum Listenpreis; mit Abo zahlst du ihn nicht). Das Zeichen ≈ heißt: Schätzung, kein genauer Wert. Fürs Cache-Lesen rechnet BroStatus mit dem Faktor des Modells, das du gerade nutzt (laut Anthropics Preisliste, Stand 29.10.2026: 0,05 bei Opus 5.5 und Sonnet 5.5, 0,025 bei Fable 5.1 und Mythos 5.1, sonst 0,1). Kennt BroStatus das Modell nicht, nimmt es 0,1; bei neuen Modellen kann der Faktor abweichen.
Töne (nur Desktop-App)	Eigene Zeile mit Leuchte (**grün** an, **rot** aus) und An/Aus, Standard an: auffälliger Ton bei „wartet auf dich“, leiser Ton bei „fertig“ nach längerer Arbeit. Unter „Mehr“ wählst du je Anlass einen von drei Tönen, „Fertig-Ton ab Dauer“ und „Ton wiederholen“ (Aus, alle 2 oder alle 5 Minuten): Ein verpasster Ton kommt höchstens 3-mal wieder, bis du schreibst, freigibst oder auswählst. Mit Probe und Test-Knöpfen. „Lautstärke der Töne“ hat die Stufen 25, 50, 75 und 100 % (Standard 50 %) und gilt für alle Töne, auch eigene; sie wirkt unter Windows, auf Mac und Linux spielt die App den Ton in ihrer eigenen Lautstärke. Im Terminal bleibt die Terminal-Glocke.
· nächste Nachricht ≈ … Token (unter „kontext“, nur im Panel)	Was die nächste Nachricht laut Cache-Zustand kostet: *warm* etwa ein Zehntel (je nach Modell weniger) des Kontexts, *kalt* der Neuaufbau. Eine Schätzung; ohne Cache-Daten keine Zahl.

Farben im Panel

- ● **warm**: alles gut, der Kreis zeigt die Restzeit.
- ● **läuft bald ab**: unter 10 min (1-h-Cache) bzw. 1 min

- ● **läuft gleich ab**: unter 2 min bzw. 30 s, ● **kalt**: abgelaufen
- Kreise: grün ab 80 %, gelb ab 50 %, sonst rot

Claude Code kompaktiert nach rund 1 Stunde Leerlauf von selbst, abschalten mit `"idleCompaction": false` in der `settings.json`. Cache-Dauer festlegen: `"promptCacheTtl": "1h"`.

Fertig- und Warte-Signale

Damit du nicht ständig nachschauen musst.

Fenstertitel

Der Titel des Terminal-Fensters zeigt den Status, auch wenn es im Hintergrund liegt:

```
◆ mein-projekt · ⚙ Bash npm test
◆ mein-projekt · ⏸ wartet auf dich
◆ mein-projekt · ◔ arbeitet im Hintergrund (2)
◆ mein-projekt · ✓ fertig (3m 20s)
```

Ton und Meldung im Terminal

Wenn Claude auf dich wartet, oder wenn eine Antwort mindestens 15 Sekunden gedauert hat:

```
💎 ✓ Claude ist fertig – mein-projekt (3m 20s)
Claude wartet auf dich (mein-projekt)
```

Die Grenze stellst du mit `notify_min_secs` ein.

Telegram über einen eigenen Bot

Optional. Praktisch, wenn du Claude vom Handy steuerst. Die Nachricht enthält den Projektnamen und die Meldung (fertig mit Dauer, oder wartet auf dich). Den Anfang von Claudes Antwort (400 Zeichen) und den Rechnernamen schickt BroStatus nur, wenn du sie in der `config.json` einschaltest (`telegram_include_answer`, `telegram_include_host`, Standard aus).

Einrichten: siehe Einstellungen.

Du willst deine eigenen Töne nutzen?

Geht unter Windows in der Desktop-App. Deine Töne bleiben bei dir, ein Update löscht sie nicht.

1. Lege im BroStatus-Ordner `C:\Users\<Name>\.claude\brostatus` (neben `state`) einen Ordner `toene` an.
2. Leg deine Töne als WAV-Dateien hinein: `warten-<Name>.wav` für den Warnton, `fertig-<Name>.wav` für den Fertig-Ton, zum Beispiel `fertig-Gong.wav`.
3. Starte eine neue Sitzung. Unter `/bro` → „Mehr" → Töne steht „Gong" hinter den eingebauten Tönen. Beim ersten Mal ist dein Ton gleich Standard.

Nimm nur Töne, die du nutzen darfst. Unter Mac und Linux spielt BroStatus nur die eingebauten Töne.

Installation

Windows | Mac | Linux. Ein Installer pro System. Er richtet Statusline, Signale und Mod ein und legt vorher ein Backup an.



Die Installer erkennst du an diesem Icon. Unter Windows hat die `.exe` dieses Icon, auf dem Mac die App. Nach der Installation hat auch der Ordner `brostatus` dieses Icon.

Windows



BroStatus-Install.exe
Doppelklick genügt

1. `BroStatus-Install.exe` herunterladen
2. Doppelklicken
3. Bei „Der Computer wurde durch Windows geschützt": **Weitere Informationen**, dann **Trotzdem ausführen**
4. Fehlt Python, installiert das Fenster es selbst (1–2 Minuten)
5. Ordner wählen oder `Enter` drücken
6. Bei „✓ Fertig!" eine Taste drücken
7. Claude Code neu starten, am besten im **Windows Terminal**

Als Ersatz gibt es `BroStatus-Install.cmd`, die dasselbe tut. Sie zeigt das normale CMD-Symbol.

Mac



BroStatus-Install.app
in `BroStatus-Mac.zip`

1. `BroStatus-Mac.zip` herunterladen und per Doppelklick entpacken
2. Beim ersten Mal **Rechtsklick** auf `BroStatus-Install.app` → **Öffnen** → **Öffnen**
3. Kein Öffnen-Knopf? **Systemeinstellungen** → **Datenschutz & Sicherheit** → **Dennoch öffnen**
4. Fehlt Python: im Terminal `xcode-select --install`, dann nochmal
5. Bei „✓ Fertig!" `Enter` und Claude Code neu starten

Linux

$_

brostatus-install.sh
Skript für das Terminal, ohne eigenes Icon

1. `brostatus-install.sh` auf den Rechner kopieren, z. B. vom PC aus (NAME = dein Benutzer, SERVER = Name oder IP-Adresse des Servers):

```
scp brostatus-install.sh NAME@SERVER:~/
```

2. Starten:

```
bash brostatus-install.sh
```

3. Fehlt Python: `sudo apt install -y python3`, dann nochmal
4. Claude Code neu starten

Die Statusline siehst du nur im Terminal, also per SSH. Über Remote Control vom Handy bekommst du stattdessen die Telegram-Meldung.

ORDNER

Standard ist `~/.claude/brostatus`, unter Windows `C:\Users\<Name>\.claude\brostatus`. Gibst du einen anderen Ordner an, landet alles darin im Unterordner `brostatus`, und Updates bleiben dort. Unter Mac und Linux: `--ziel <ordner>`.

EINSTELLUNGEN

Die `settings.json` von Claude Code liegt normalerweise in `~/.claude` (oder in dem Ordner, den `CLAUDE_CONFIG_DIR` angibt). Der Installer trägt dort Statusline, Hooks und Mod ein und legt vorher ein Backup an.

Prüfen, ob alles läuft

Nach dem Neustart in drei Schritten.

Erst Claude Code starten, dann /bro: `/bro` und `/plugin` sind Befehle *in* Claude Code, nicht in PowerShell oder im Terminal. Terminal normal öffnen (nicht als Administrator), in den Projektordner wechseln (`cd D:\mein-projekt`), `claude` eingeben. Beim ersten Start meldet sich Claude Code einmal im Browser an. In der Desktop-App einfach eine neue Sitzung starten.

1 · STATUSLINE

Unten erscheinen Diamant, „○ bereit“ und die Infozeilen:



```
○ bereit · opus/high
kontext ▮▮▮▮▮▮▯▯▯▯▯▯▯▯▯▯ 46% 92k/200k
cache   ◷ 54:58 1h   treffer 99% letzte · 90% sitzung   limit · S | 5h / 23%  · W | 7d / 71%   #tools_chan…
aktiv   –
dateien –
```

2 · MOD GELADEN

`/plugin` eingeben. Unter den Tabs steht „1 mod active · bro-status“ (mit weiteren eigenen Mods eine höhere Zahl).

```
> /plugin
1 mod active · bro-status
```

Ausschnitt, nachgebaut

3 · KNÖPFE

Claude bitten, dir zwei Fragen als nummerierte Liste zu stellen. Über dem Eingabefeld erscheint „◆ Antworten“.

```
◆ Antworten  Knopf drücken  Send  Hide
Farbe wählen
Logo links oder rechts
```

Ausschnitt, nachgebaut

Einstellungen

In `~/.claude/brostatus/config.json`. Änderungen wirken sofort, ohne Neustart (nur der eigene Kurs wird beim Start der Mod gelesen).

SCHLÜSSEL	STANDARD	BEDEUTUNG
`notify_min_secs`	15	Ab so vielen Sekunden Arbeit kommt eine Fertig-Meldung
`terminal_notify`	true	Ton und Desktop-Benachrichtigung
`window_title`	true	Status im Fenstertitel
`telegram_bot_token`	leer	Token deines Bots für Telegram-Meldungen
`telegram_chat_id`	leer	Deine Chat-ID für Telegram
`logo`	true	Diamant links anzeigen
`colors`	"truecolor"	Auf "256" stellen, falls Farben falsch aussehen
`ctx_stages`	[50, 70, 85]	Grenzen der Kontext-Ampel in % (gelb, orange, rot)
`files`	"compact"	"compact" = eine Zeile, "tree" = Dateibaum, "off" = aus
`tree_max_lines`	8	Höchstens so viele Zeilen für den Dateibaum
`max_items`	5	Einträge pro Art in der Aktiv-Zeile

Telegram mit einem eigenen Bot einrichten

1. In Telegram mit `@BotFather` einen eigenen Bot anlegen und den Token notieren
2. Deinem Bot in Telegram eine beliebige Nachricht schicken
3. Im Browser öffnen, `<TOKEN>` durch den Bot-Token ersetzen: `https://api.telegram.org/bot<TOKEN>/getUpdates`

4. Unter `"chat":{"id": …}` steht deine Chat-ID
5. Token und Chat-ID in die `config.json` eintragen:

```
"telegram_bot_token": "123456:ABC...",
"telegram_chat_id": "deine-chat-id"
```

Updates und Probleme lösen

Für ein Update startest du einfach die neue Installer-Datei. Deine `config.json` bleibt erhalten.

PROBLEM	LÖSUNG
`claude` wird nicht erkannt	Claude Code ist installiert, aber nicht im PATH. Einmal in PowerShell ausführen, dann PowerShell neu öffnen: `[Environment]::SetEnvironmentVariable('Path', [Environment]::GetEnvironmentVariable('Path','User') + ";$env:USERPROFILE\.local\bin", 'User')`
`/bro` wird nicht erkannt (rote Meldung in PowerShell)	Du bist noch in PowerShell. Erst `claude` starten, dann `/bro` in Claude Code eingeben
In der Desktop-App nimmt `Enter` nach `/bro` einen anderen Vorschlag (z. B. einen Skill)	Das Vorschlagsmenü zeigt nach `/bro` nur Skills, nicht den Mod-Befehl. Erst `Esc` drücken, dann `Enter`. In einer frischen Sitzung startet die Mod erst mit deiner ersten Nachricht
„Browser didn't open? … Paste code here“	Einmalige Anmeldung von Claude Code: Link öffnen, mit deinem Claude-Konto anmelden, den angezeigten Code ins Terminal einfügen (Rechtsklick), `Enter`. Den Code nirgends sonst teilen
Claude Code startet in `C:\WINDOWS\system32`	PowerShell nicht als Administrator öffnen und vorher mit `cd` in deinen Projektordner wechseln
Kästchen statt Diamant	Windows Terminal statt des alten cmd-Fensters nutzen, oder `"logo": false` setzen
Farben sehen komisch aus	`"colors": "256"` in der `config.json` setzen
Keine Statusline	Claude Code neu starten und dem Ordner vertrauen (Trust-Abfrage bestätigen)
`/plugin` zeigt bro-status nicht	Version prüfen mit `claude --version`, getestet ist 2.1.310 (ältere Versionen nicht geprüft). Sonst `claude update`
Knöpfe ohne Maus erreichen	`Strg+X`, dann `Tab` und `Enter`
Python fehlt unter Windows	Von python.org installieren, Haken bei „Add python.exe to PATH“ setzen, Installer nochmal starten
Python fehlt unter Linux	`sudo apt install -y python3`
„Da ist etwas schiefgelaufen“	Screenshot in deine Claude-Code-Sitzung einfügen und um Hilfe bitten oder als Issue melden

Statusline einzeln testen

Erscheint der Diamant mit „○ bereit“, ist BroStatus wahrscheinlich in Ordnung, und das Problem liegt eher bei Claude Code. Der Befehl gilt für Mac und Linux; unter Windows nimmst du PowerShell mit `python` statt `python3`.

```
echo '{"session_id":"test","model":{"display_name":"Opus"}}' |
python3 ~/.claude/brostatus/statusline.py
```

Deinstallieren

Mac und Linux:

```
python3 ~/.claude/brostatus/install.py --uninstall
```

Windows (PowerShell):

```
python $HOME\.claude\brostatus\install.py --uninstall
```

Trägt alles aus der `settings.json` aus, mit Backup. Der Ordner `brostatus` bleibt liegen, mit deiner `config.json` (darin steht, falls eingerichtet, dein Telegram-Token). Lösche ihn von Hand, wenn du ihn nicht mehr brauchst.

BroStatus ist ein Projekt von DiamondSash (github.com/diamondsash/bro-status). Es ist für Claude Code gemacht, braucht keine eigenen Server und verbraucht für die Anzeige keine Tokens. Lizenz: MIT mit Commons Clause (kostenlos nutzen, nicht verkaufen). Nicht mit Anthropic verbunden. Claude und Claude Code sind Marken von Anthropic.